

Deep Learning Midterm Project Report

Che Hung Liao
cl8172@nyu.edu

Eleftherios Komvopoulos
ek4538@nyu.edu

Abstract

This report presents our approach to a deep learning task using modern neural network methods. We focus on building a competitive model under practical training-time constraints, compare multiple configurations, and summarize the design and optimization choices that contributed most to performance.

1 Introduction

This project focuses on solving a deep learning task using modern neural network approaches. The goal is to build a model that achieves strong performance on the given dataset while exploring different architectural and training choices.

Due to the computational cost of training, we emphasize iterative experimentation and practical improvements over exhaustive search. We evaluate different configurations and analyze what contributes most to performance.

1
2
3

2 Dataset

[TO BE FILLED]

We describe the dataset, preprocessing steps, and any augmentation techniques applied.

3 Model Description

We build on the Qwen2.5 series as our base model family, which provides strong general-purpose capabilities and a range of parameter sizes suitable for experimentation on limited hardware. For this task we chose the Qwen/Qwen2.5-Coder-XB-Instruct variants because they are optimized for

code-oriented reasoning and instruction-following behavior, aligning well with the debugging, code-related structured output, and prompt-driven workflow required by the competition setting. We initially evaluated 1.5B, 3B, and 7B parameter models; however, the 7B version consistently pushed GPU memory to near out-of-memory limits, so we ultimately standardized on 3B and 1.5B models as the best balance between capacity and stable training/inference.

4 Experimentation

We conducted multiple experiments to improve model performance through iterative refinement.

Hyperparameters:

- Learning rate: [fill if known, e.g., 1e-3, 1e-4]
- Batch size: [e.g., 16, 32]
- Number of epochs: [range or estimate]

Experimented Variations:

- Baseline model configuration
- Adjustments to model architecture
- Different training settings (learning rate, batch size)

Due to long training cycles, we focused on testing a limited number of meaningful configurations rather than performing an exhaustive search. We prioritized changes that were most likely to impact performance based on intuition and prior experience.

5 Results

The final model achieved improved performance compared to the baseline. We observed that tuning hyperparameters, particularly learning rate and

¹Code repository: github.com/shanpig/dl-kaggle-midterm.

²Model weights: [ADD YOUR LINK HERE]

³We used GPT and Cursor for debugging and implementation assistance.

Model	Score
Baseline	[XX.X]
Final Model	[XX.X]

Table 1: Comparison between the baseline and final model performance.

model configuration, had a noticeable impact on results.

In some cases, increasing model complexity did not yield proportional improvements, indicating diminishing returns.

6 Conclusion

In this project, we explored different modeling approaches and training strategies to improve performance on the task.

We found that careful tuning of hyperparameters and iterative experimentation were key to achieving better results. However, training time posed a significant constraint, limiting the number of experiments that could be conducted.

Some approaches did not lead to improvements, highlighting the importance of targeted experimentation rather than blindly increasing complexity.

In future work, we would implement better experiment tracking and explore more efficient training methods to allow broader exploration of the design space.